Ders 10 Lab Çalışma Özeti

Ders 10 Lab Çalışma Özeti

Genel Konular

- Process Synchronization'a Giriş
  - Modern OS'ler thread yapısı üzerine kurulu (1:1 mapping).
  - Process senkronizasyonu içinde thread senkronizasyonu da girer; ikisi birlikte düşünülmelidir.
  - Bu ders kritik bölge, senkronizasyon çözümleri (yazılımsal, donanımsal), senkronizasyon mekanizmaları (mutex, semafor, monitör) ve klasik senkronizasyon problemlerini kapsar.
- Critical Section (Kritik Bölge) Problemi
  - Birden fazla process ortak bir alana (global değişken, veri yapısı, dosya) eşzamanlı erişirse tutarsızlıklar oluşur.
  - Aynı processler aynı kodla her seferinde farklı sonuç üretiyorsa, elde edilmesi gereken sonuç net, kesin, değişmez olmalı.
  - Senkronizasyon bunun içindir: race condition oluşmasını engellemek, çalışmayı belirli bir sıraya senkronlamak.
- Race Condition (Yarış Durumu)
  - Örnek: counter değişkeni üzerinde iki process'in arttırma-eksiltme işlemi. İkisi de okur (5), biri arttırır (6), diğeri eksiltir (4); sonuç farklı olabilir.
  - Bölünmeler, kesintiler olduğunda iç içe geçmiş çalışma sonucu farklı sonuçlar çıkar.
  - Race: yarış demek. Yarışı kimin kazanacağı belirsiz; her çalıştırmada farklı sonuç.
- Kritik Bölgede Modifikasyon
  - Ortak kaynak üzerinde yapılan modifikasyonun (arttırma, eksiltme, çarpma, bölme, değiştirme) yapıldığı bölge kritik bölgedir.
  - Bir process bu bölgede iken kesintiye uğramamalı (rahatsız edilmemeli).
  - Fiziksel olarak interrupt'ları disable etmek çözüm değildir (OS interrupt-driven çalışır).
- Çözüm Yöntemlerinin Seviyeleri
  1. Yazılımsal çözümler (Peterson).
  2. Donanımsal çözümler (memory barriers, hardware instructions).
  3. Üst seviye yapılar (mutex lock, semafor, monitör).
- Peterson Çözümü
  - İki process'in kritik bölgeye girmesini yazılımsal olarak engelleyen algoritma.
  - `flag[i]` = process i kritik bölgeye girmek istiyor mu?
  - `turn` = sıra kimde?
  - Basit ama ölçeklenebilir değil.
- Donanımsal Çözümler
  - **Memory Barriers**: Memory modeline uygun sıralama garantisi.
  - **Hardware Instructions**: Atomik (kesintisiz) instruction'lar.
    - `test_and_set(&lock)`: lock'u 1 yapar, eski değeri döndürür.

    * `compare_and_swap(&value, expected, new)`: değer expected'a eşitse new ile değiştirir, eski değeri döndürür.
  - Bu instruction'lar kesilemez.

- Mutex Locks
  - Basit kilit mekanizması. `acquire()` ile kilit al, `release()` ile bırak.
  - Dezavantaj: busy waiting (kilit açılana kadar CPU harcamak).
  - Kilit açılana kadar spin eder.

- Semaforlar
  - Tamsayı değer + bekleme kuyruğu.
  - `wait(P)`: Değer ≤ 0 ise bloklan, > 0 ise 1 azalt.
  - `signal(V)`: Değeri 1 arttır, bekleyeni uyandır.
  - **Binary Semaphore (Mutex)**: Değer 0 veya 1.
  - **Counting Semaphore**: Değer 0-N.

- Semafor Örneği (C Kodu)

```c
sem_t mutex;
sem_init(&mutex, 0, 1);  // 1 ile başlat
sem_wait(&mutex);
// kritik bölge
sem_post(&mutex);
```

- Mesaj Kuyruğu (Message Queue)
  - Process'ler arası mesaj alışverişi için.
  - Fonksiyonlar: `mq_open`, `mq_send`, `mq_receive`, `mq_close`, `mq_unlink`.
  - Özellikler: max message, message size, current message count.
  - Mesaj formatı: descriptor, pointer, length, priority.
  - Yüksek öncelikli mesaj önce okunur.
  - Kalıcıdır: kuyruk kapatılsa bile mesajlar silinmez.
  - Writer ve reader aynı anda kuyruğu açmak zorunda değildir.

- Shared Memory (Paylaşımlı Bellek)
  - Fonksiyonlar: `shm_open`, `shm_unlink`, `mmap`, `munmap`, `ftruncate`, `shm_close`.
  - `mmap`: Paylaşılan belleği process'in adres alanına eşler.
  - `ftruncate`: Dosyanın boyutunu ayarlar.
  - Protection: PROT_READ, PROT_WRITE, PROT_EXEC, PROT_NONE.
  - Flag'ler: MAP_SHARED, MAP_PRIVATE, MAP_FIXED.
  - Fork ile aktarılamaz; tekrar `shm_open` gerekir.

Hocanın Özellikle Vurguladığı Kısımlar

- Semafor Kullanım Kuralları
  - Hoca vurgular: "Semaforları çok güzel, çok iyi. Ancak semaforları kullanırken, kullanım kurallarına dikkat etmez ise otomatikman problemlere sebep oluyoruz. Şeyi hatırlayın, size bunu üzerine basa basa söyledik. İşletim sistemleri Deadlock için çok özel bir çözüm üretmiyorlar."
  - "Çünkü Deadlock çözünmesiyle alakalı gerçekleştirecek olan algoritmaların karmaşıklığı yüksek. Onlar için harcanacak olan işlem gücü, boşa harcanmış olan bir işlem gücü."
- Sinyal/Wait Yanlış Sıra
  - "Eğer aynen bu noktada signal, weight, mutex derseniz bu sırayla kullanırsanız, critical section, herkesin girebileceği bir critical section'a dönüşüyor. Yani critical section olmaktan çıkıyor. Weight, mutex, weight, mutex derseniz siz girebiliyorsunuz critical section'a ama siz çıkamıyorsunuz."
  - "Veya signal'ın bir tanesini veya iki tanesini unutmak ve kullanmamak daha sıkıntılı başka şu anda doğrudan aklımıza gelmese bile problem olacak durumlara yol uçuyor."
- Monitör Yapısı

  - "Monitorda herhangi bir anda sadece bir proses aktif olabilir. O zaman bakın monitorun girişinde bir kuyruk var. Sadece bir tanesi aktif olduğuna göre, ben de bu kuyruğu sıralı bir kuyruk olarak modelleyebilirim."
  - "Yani monitorun yapısına girdiğinizde bununla ilgili ayrıntıları incelediğinizde aslında monitor bir klas oldu. O monitor kullanılarak synchronizasyon sağlanacağı zaman da o klasdan bir instance, bir objekt oluşturulduğunu göreceğiz."
- Programcıya Bırakılan Sorumluluk
  - "Burada tabi ki vurgun nereye yapılıyor? Program geliştiren kişinin bunun çok dikkatli kullanmasına yapılıyor. Şimdi sizler bu iş için mühendisi olarak, bunun eğitimini aldığınız için tabi ki her şart altında weight ve signal mekanizmasını, ve ki bounded buffer problemini gördünüz, reader's writers problemini gördünüz, dining closed buffer problemini gördünüz, her şart altında doğru kullanmanız lazım."

Kısa Tekrar Notları

- Race condition: eşzamanlı erişim sonucu belirsizlik.
- Critical section: paylaşılan kaynağa erişim bölgesi.
- Mutual exclusion: aynı anda sadece bir process.
- Test-and-set, compare-and-swap: atomik donanım instruction'ları.
- Mutex lock: basit kilit (busy waiting).
- Semaphore: wait(P) ve signal(V) ile senkronizasyon.
- Monitör: üst seviye senkronizasyon yapısı.
- Bounded buffer, readers-writers, dining philosophers: klasik problemler.
- Message queue: mq_open, mq_send, mq_receive.
- Shared memory: shm_open, mmap, ftruncate.

Detaylı Açıklamalar

Ders 10 Lab, proses senkronizasyonu konusunu yazılımsal ve donanımsal bakış açısıyla ele alır. Bu ders, IPC konusunun devamıdır: process'ler arası veri paylaşımında tutarlılık nasıl sağlanır?

Process'ler ortak kaynaklara (global değişken, veri yapısı, dosya) eriştiğinde tutarsızlıklar oluşabilir. Race condition örneği verilir: iki process aynı `counter` değişkenini değiştirmek istiyor. İkisi de okur (5), biri arttırır (6), diğeri eksiltir (4); sonuç farklı olabilir. Bu tanımlı değildir. Senkronizasyon bunu engellemek için vardır.

Kritik bölge (critical section) kavramı detaylı açıklanır. Process'in paylaşılan kaynağa erişip değişiklik yaptığı kod bölgesi kritik bölgedir. Process bu bölgede iken kesilmemelidir. Fiziksel olarak interrupt'ları disable etmek bir çözüm gibi görünür ama OS interrupt-driven çalıştığı için bu kabul edilemez.

Çözüm yöntemleri üç seviyede incelenir:

Yazılımsal Çözümler: Peterson çözümü iki process için tasarlanmış yazılımsal bir algoritmadır. `flag[]` dizisi (her process kritik bölgeye girme isteğini belirtir) ve `turn` değişkeni kullanılır. Basit ama ölçeklenebilir değildir.

Donanımsal Çözümler: Modern işlemciler atomik (kesintisiz) instruction'lar sağlar. `test_and_set` hedefi 1 yapar ve eski değeri döndürür. `compare_and_swap` üç parametre alır. Bu instruction'lar atomiktir; kesilmezler.

Üst Seviye Yapılar: Mutex lock'lar (basit ama busy waiting), Semaforlar (wait/signal ile senkronizasyon), Monitörler (klas-tabanlı, paylaşılan değişken ve prosedürler içerir).

Üç klasik senkronizasyon problemi detaylı anlatılır: - **Bounded-Buffer**: Producer-consumer. 3 semafor: mutex, full, empty. - **Readers-Writers**: Veritabanı senaryosu. rw_mutex ve mutex. - **Dining-Philosophers**: 5 filozof, 5 çubuk. Deadlock ve starvation riski.

Monitör kavramı detaylı açıklanır. Monitör, bir sınıf (class) benzeri yapıdır: paylaşılan veri + bu veri üzerinde çalışan prosedürler içerir. Monitörde herhangi bir anda sadece bir proses aktif olabilir; bu mutual exclusion'ı otomatik sağlar. Koşul değişkenleri (condition variables) ile prosesler senkronize edilebilir. `wait` ve `signal` operasyonları monitör için tanımlıdır. `wait(x)`

ile proses bloklanır, başka proses `signal(x)` ile uyandırır. Monitör, semaforların kullanım hatalarını ortadan kaldırır.

C kod örnekleri üzerinden mesaj kuyruğu ve paylaşımlı bellek kullanımı gösterilir. `mq_open`, `mq_send`, `mq_receive`, `mq_close` fonksiyonları ile message queue kullanımı; `shm_open`, `mmap`, `ftruncate` fonksiyonları ile shared memory kullanımı açıklanır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.